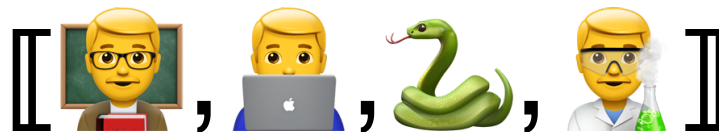


Lecture Notes for **Machine Learning in Python** **aka (Neural Networks I)**



Professor Eric Larson
Introduction, Syllabus, Data Types

Class Logistics and Agenda

- Agenda:
 - Course Overview
 - Introductions/Cards
 - Syllabus
 - What is Machine Learning?
 - Types of Data
 - Numpy/Pandas Demo
- My approach to this course:
 - Programming
 - Math
 - **Applications** and **Analytics**

Class Overview, by topic

Table Data
Visualization

Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Linear and
Logistic
Regression

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Neural Networks
and Back Prop.

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Wide and Deep
Networks

Convolutional
Networks

Sequential
Networks

Keras, Tensorflow, **Torch**
Intuition, Detailed implement.

Large Language
Models (if time)

Hugging Face
Intuition only

Different than Previous Years

Class Overview, by assignment

- **Flipped Module One:** Linear Regression
- **Lab One:** Visualize data and extract features
- **Lab Two:** Develop Logistic Regression in Style of Scikit
- **Flipped Module Two:** Back Propagation
- **Midterm Exam:** Table Data, Logistic Regression, Basic NN
- **Flipped Module Three:** Cross Validation
- **Lab Three:** Use Deep Learning APIs (Torch)
- **Lab Four:** CNN Transfer Learning
- **Lab Five:** Transformer Transfer Learning
- **Final Exam:** Comprehensive

All Assignments posted on Canvas (with Rubric)
Labs are Team Assignments

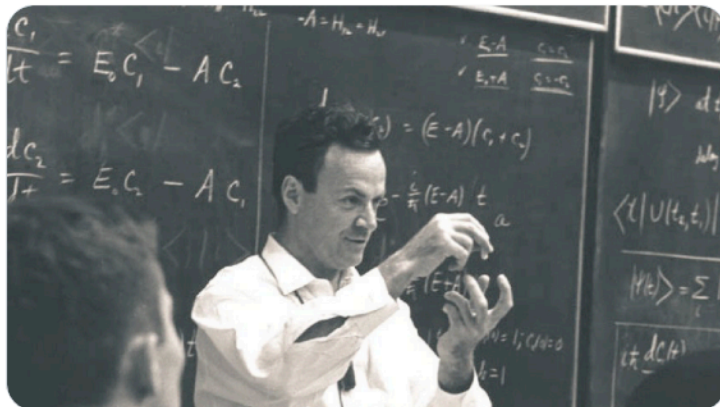
Introductions & Course Syllabus



Richard Feynman @ProfFeynman · 12h

Don't just teach your students to read.

- Teach them to question what they read, what they study.
- Teach them to doubt.
- Teach them to think.
- Teach them to make mistakes and learn from them.
- Teach them how to understand something.
- Teach them how to teach others.



Richard Feynman @ProfFeynman · 21h

You cannot get educated by this self-propagating system in which people study to pass exams, and teach others to pass exams, but nobody knows anything.

You learn something by doing it yourself, by asking questions, by thinking, and by experimenting. 🧠



Introductions

- Me
 - Dr. Larson 👍
 - Prof. Larson 👍
 - PhD students: (Eric) 🙄
 - Other 🙄
- You
 - Name & Department
 - Grad/Undergrad/other?
 - Something true or false
 - Write your name on Back of card (**if not already**)

Quick introductions (depends on Class Size)

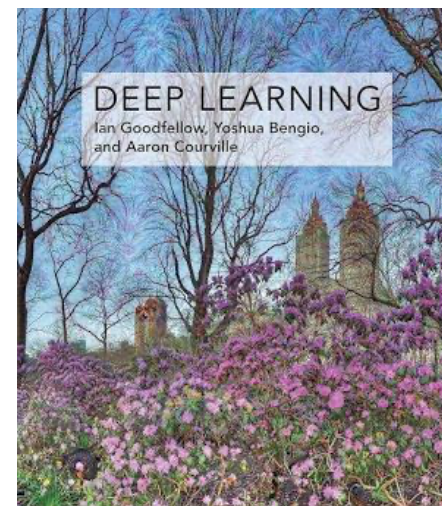
FAQ

- **Recommended Texts:**
 - **Neural Networks and Deep Learning**, by Michael Nielsen
(<http://neuralnetworksanddeeplearning.com/index.html>)
 - **Deep Learning** by Goodfellow et al.
(<https://www.deeplearningbook.org/>)
- Use Canvas for posted course material
- Prerequisites:
 - Linear algebra & multivariate calculus
 - Programming / data structures
 - Basic statistics and random variables
- Version of **python**, see **GitHub**
 - Install through **pip** (or **conda**)
 - Use **conda** environments
 - JupyterLab or **jupyter notebook**
- Most Used Libraries: Numpy, Pandas, Scikit-Learn, Matplotlib, Seaborn, Tensorflow / Torch
- Use OIT Data Science Workshops

Neural Networks and Deep Learning

Neural Networks and Deep Learning is a free online book. The book will teach you about:

- Neural networks, a beautiful biologically-inspired programming paradigm which enables a computer to learn from observational data
- Deep learning, a powerful set of techniques for learning in neural networks



Canvas Syllabus

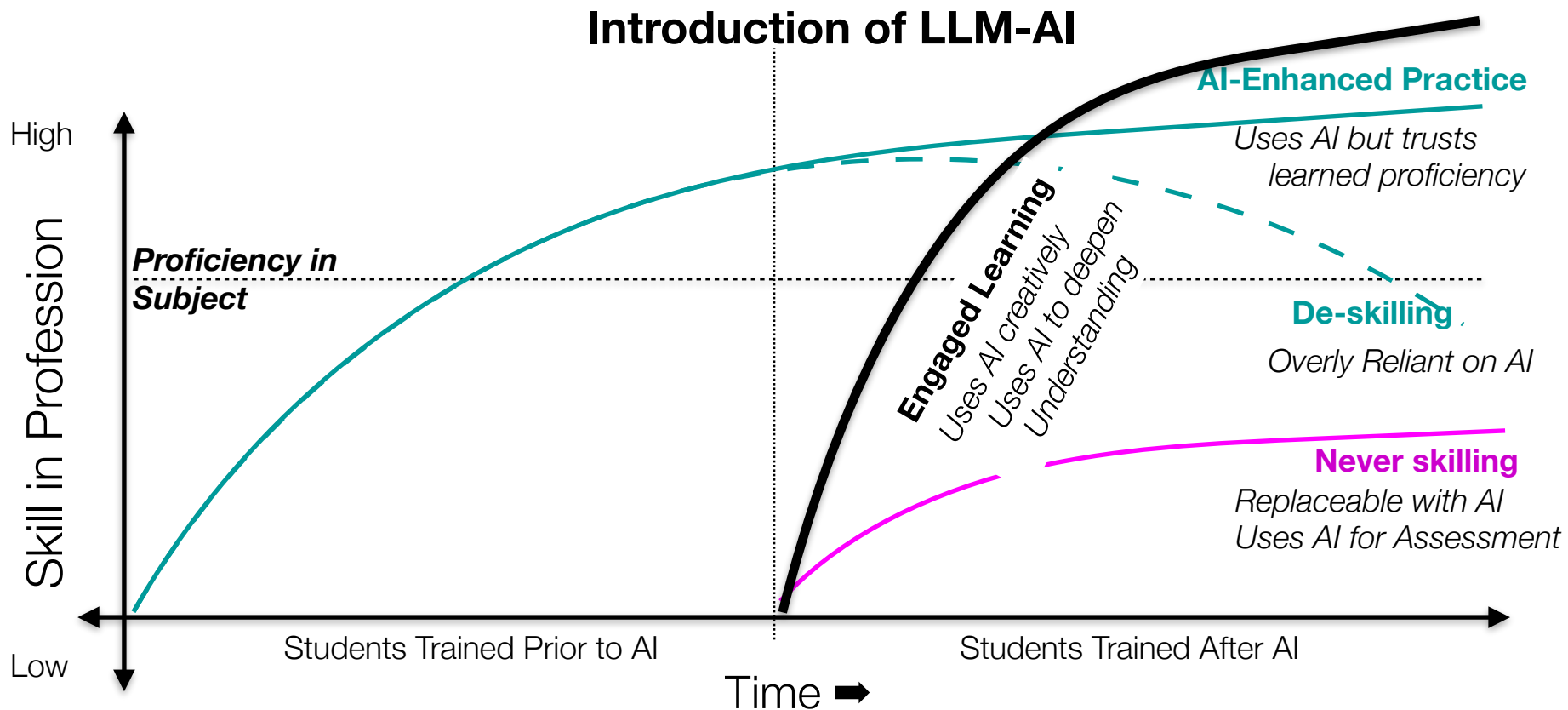
- Course Schedule
- LLM Policy
- Videos / Slides
- Lab Assignments x5
- Flipped Assignments x3
- Exams (Mid term and Final)
- Grading Rubrics
- Participation
- Difference between 5000 and 7000

How will participation be graded?

- Participation will be graded in the course:
 - **Distance students** will answer these questions via **canvas upload** (same for Zoom)
 - upload “over” the last submission
 - must upload the questions each week for full credit
- In Class Students:
 - Live question answering (mostly attendance):
 - Do you think this will work?
 - A: **Yes** this is going to work.
 - B: This is **not** going to work.
 - C: Yes. And I added my name to the card.
 - D: I held the card in the wrong position.

Large Language Models (Instruct Models)

- LLMs are incredible tools
- But can minimize learning (unless purposeful)



LLM Guided Example

[1.5 points] Business Understanding. In this portion of the rubric, you will be developing a business understanding section. **Prompt an LLM to generate a business understanding summary.** You should **design a prompt that** helps you to:

- Describe the purpose of the data set you selected and how it was collected.
- Describe the prediction task for your data.
- Answer why are third parties interested in the result, and what evaluation measure can be used to prove to third parties that your model works.
- Describe what level of performance would be useful and why.

In your notebook, you should provide the prompt you created and provide the output from the LLM. Points will be given for well-designed prompts. **Identify at least two inaccuracies, omissions, or generic statements in the LLM response that should be corrected. Provide a refined version in your own words that corrects these problems.**

...

[1.5 points] Data Loading. ***Without the assistance of an LLM,*** load the dataset and appropriately define data types. What data type should be used to represent each data attribute? Justify representation choices (categorical, ordinal, numeric). For datasets with a large number of attributes, only discuss a subset of relevant attributes. Also summarize missing value in each feature.

Machine Learning Overview



What is Machine Learning?



The 1956 Dartmouth Summer Research Project on **Artificial Intelligence**, organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon, recognized as the founding event of AI.

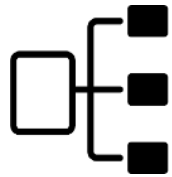
The six-week workshop brought together researchers to explore the idea that machines could simulate human intelligence, language, and problem-solving, officially coining the term "artificial intelligence"

Machine Learning is a Subset of Artificial Intelligence, and continues to evolve as capability expands. It is often used as a tool within a larger AI system.

Problem Types in Machine Learning

- Inputs

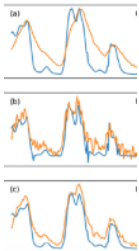
- Outputs



1.23
-0.4
...



This is a repository for my
experience in Python and
purpose.



- Categories

- Numeric Data

- Images

- Free Text

- Time Series

classification

regression

image generation

text generation

auto encoding

- Categories

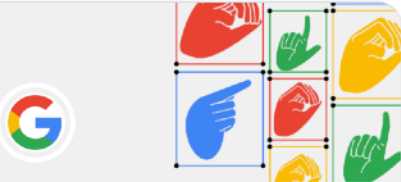
- Numeric Data

- Images

- Free Text

- Time Series

Problem Types in Machine Learning



Google - American Sign Language Fingerspelling...

Train fast and accurate American Sign...

Research · Code Competition

1269 Teams

\$200,000 3 days to go



CommonLit - Evaluate Student Summaries

Automatically assess summaries writt...

Featured · Code Competition

925 Teams

\$60,000 2 months to go



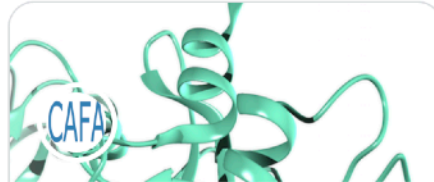
Bengali.AI Speech Recognition

Recognize Bengali speech from out-of...

Research · Code Competition

317 Teams

\$53,000 2 months to go



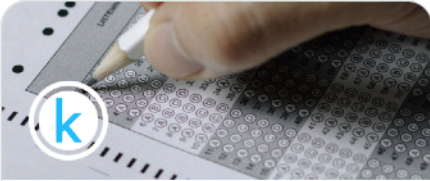
CAFA 5 Protein Function Prediction

Predict the biological function of a pro...

Research · Code Competition

1655 Teams

\$50,000 10 hours to go



Kaggle - LLM Science Exam

Use LLMs to answer difficult science ...

Featured · Code Competition

1471 Teams

\$50,000 2 months to go



RSNA 2023 Abdominal Trauma Detection

Detect and classify traumatic abdomi...

Featured · Code Competition

333 Teams

\$50,000 2 months to go



Predict CO2 Emissions in Rwanda

Playground Series - Season 3, Episod...

Playground

1401 Teams

Swag 10 hours to go



Titanic - Machine Learning from Disaster

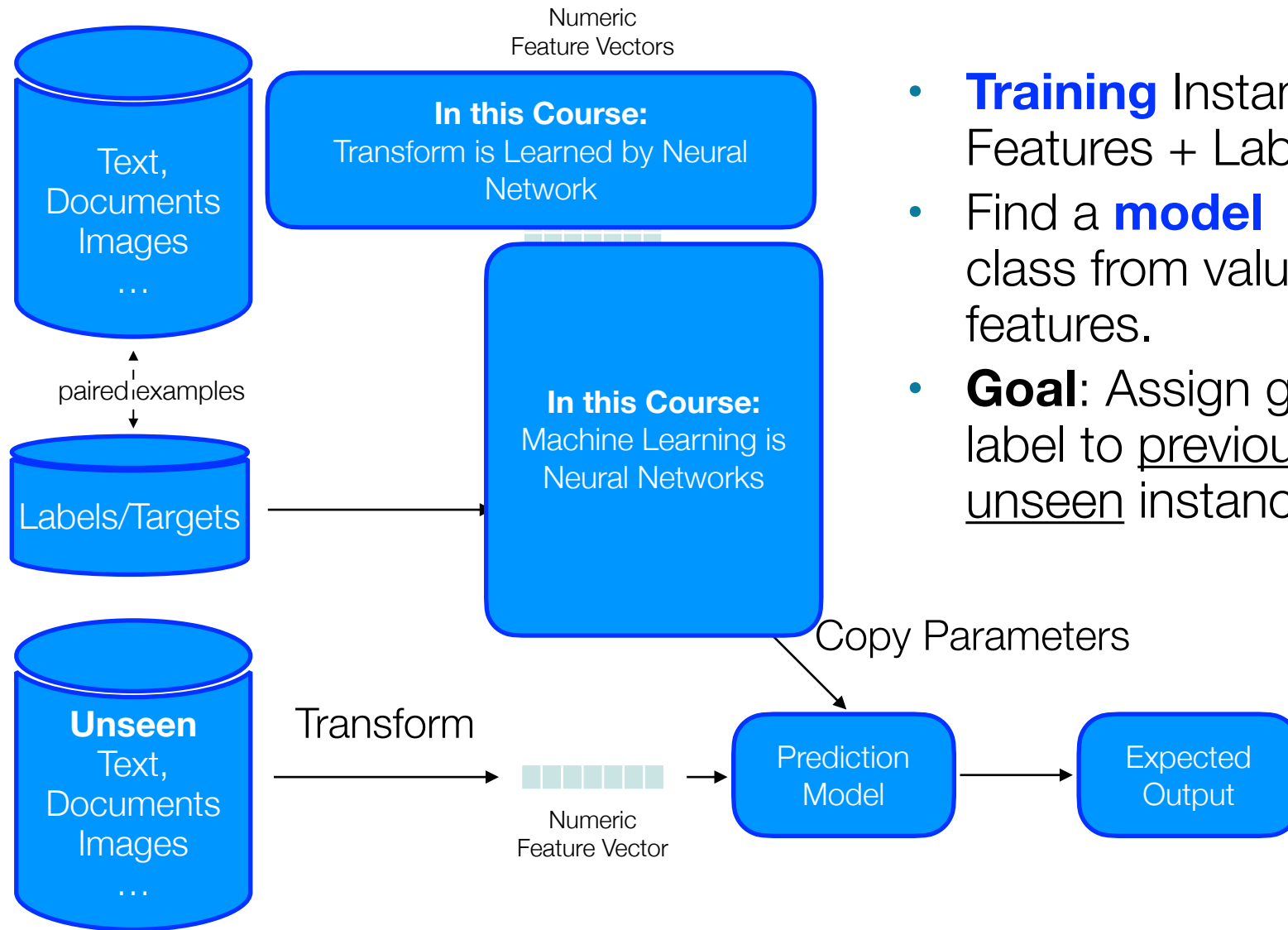
Start here! Predict survival on the Tita...

Getting Started

14897 Teams

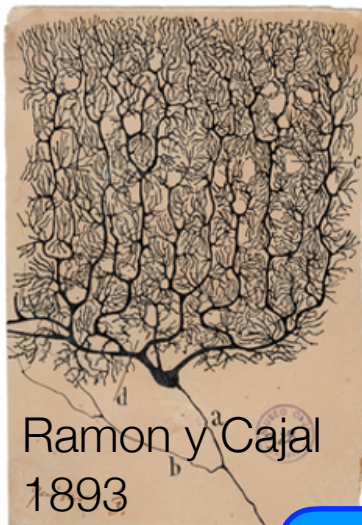
Knowledge Ongoing

Classification and Regression, Supervised



- **Training** Instances: Features + Labels
- Find a **model** mapping class from values of features.
- **Goal:** Assign guessed label to previously unseen instances

Machine Learning in This Course



Biologically Inspired

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

*Institute for Cognitive Science, CRL, University of California, San Diego, La Jolla, California 92037, USA
†Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neuron-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors². Learning becomes more interesting but

more difficult when we introduce hidden units whose actual or desired states are not specified by the task. In perception, there are 'feature analyzers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations. The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired output behavior. This amounts to deciding what those units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom, any number of intermediate layers, and a layer of output units at the top. Connections within a layer or from higher to lower layers are unidirectional, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_i , to unit j is a linear function of the outputs, y_k , of the units that are connected to j and of the weights, w_{kj} , on these connections

$$x_j = \sum_k w_{kj} y_k \quad (1)$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

A unit has a real-valued output, y_j , which is a non-linear function of its total input

$$y_j = \frac{1}{1 + e^{-x_j}} \quad (2)$$

* To whom correspondence should be addressed.

© 1986 Nature Publishing Group



Turns into Optimization of Matrices

...

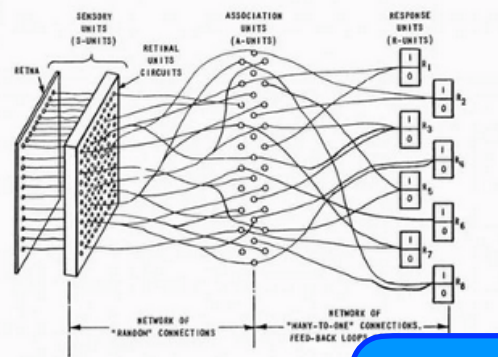
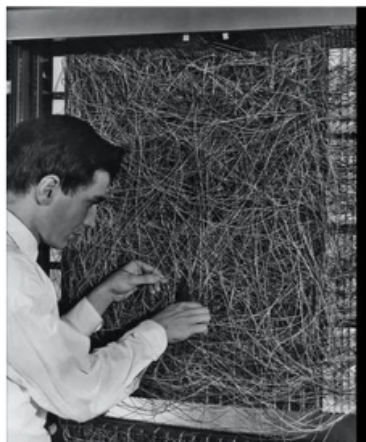
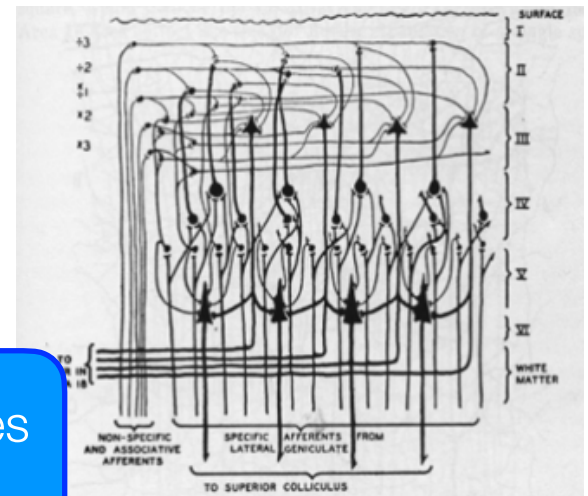


Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON

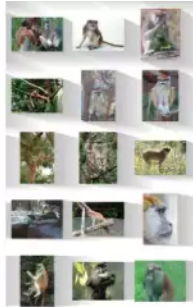


Fundamentally guides Connectivity

4: Frank Rosenblatt with his Mark I perceptron(left), and a graphical representation of it(right).
<https://data-science-blog.com/blog/2020/07/16/a-brief-history-of-neural-nets-everything-you-should-know-before-learning-1stm/>

Some Popular Datasets

ImageNet



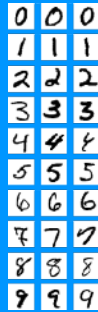
1M+

224 x 224 Color Image



1000 Classes
(prominent object)

MNIST



60k

28 x 28 Grey Image



10 Classes (digits)

Adult

# feature	original feature
1	age
2	workclass
3	final weight
4	education
5	ed_num
6	marital_status
7	occupation
8	relationship
9	race
10	sex
11	capital_gain
12	capital_loss
13	hours x week
14	country

5k

Census Demographics



Binary (salary > 50k?)

CoCo



200k Images

Large, Multi-sized Images



Location, Size, 80 Objects

Boston Housing

1. CRIM
2. ZN
3. INDUS
4. CHAS
5. NOX
6. RM
7. AGE
8. DIS
9. RAD
10. TAX
11. PTRATIO
12. B
13. LSTAT
14. MEDV

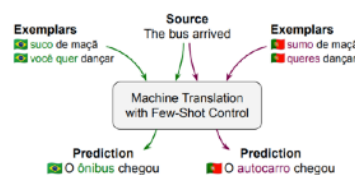
House/Neighborhood
Descriptions



House Price
\$\$

500 Examples

Translation



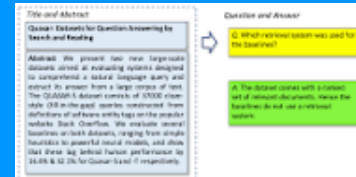
Language A



Language B

Many datasets

SQuAD



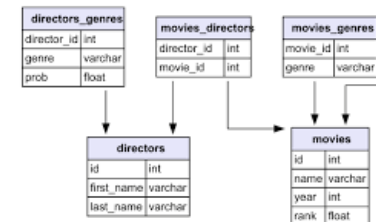
Question



Answer

100k+

Imdb



Movie/Actors/Director/+



Critic/Audience rating

50k reviews

Self Test

- **A. Classification**
B. Regression
C. Not Machine Learning
- **D. Machine Learning Generation**
- Dividing up customers by potential profitability?
- Extracting frequency of sound?

Before Next Lecture

- Before next class:
 - install python (see GitHub for Version) on your laptop
 - install anaconda distribution of python
 - use virtual environments (`conda env`)
 - then `pip install` everything (see GitHub instructions)
- Look at Python primer if you need review
 - I made ~4 hours of YouTube content...
 - <https://www.youtube.com/playlist?list=PL7IPdRN5E0YKCnVI-fvx8jOOCWVeGTsrV>

Opening Demo: Jupyter Notebooks

Most Install Instructions (Windows, Linux, and Intel-base Macs)

For operating systems that do NOT use Apple Silicon, you can use the following commands:

```
conda create --name mlenv3_12 python=3.12
```

```
conda activate mlenv3_12
```

```
pip install jupyter numpy scipy pandas matplotlib scikit-learn plotly missingno
```

```
pip install protobuf tensorflow-datasets importlib-resources
```

```
pip install torch torchvision transformers
```

Apple Silicon MacOS Install Instructions

Note: Apple M-series Macs should use a different install procedure as follows:

```
conda create --name mlenv3_12 python=3.12
```

```
conda activate mlenv3_12
```

```
python -m pip install tensorflow-macos
```

```
python -m pip install tensorflow-metal
```

```
pip install jupyter pandas matplotlib scikit-learn plotly missingno
```

```
pip install "numpy<2.0" "scipy<=1.13.1"
```

```
pip install protobuf tensorflow-datasets importlib-resources
```

```
pip install torch torchvision transformers
```



Lecture Notes for **Neural Networks I**

Professor Eric Larson
Introduction, Syllabus, Data Types